

HDOC DAY-35

Title: Square Matrix Diagonal Operations and Alternating Zig-Zag Diagonal Traversal

1. Problem Statement

Write a menu-driven C program to read a variable-length square integer array and perform the following operations using separate functions:

1. Determine whether all the elements on its main diagonal are distinct.
2. Find the sum of its main diagonal elements.
3. Display its elements using a diagonal traversal. Begin with the element in the top-left corner and traverse successive diagonals alternately in the upward-right and downward-left directions.
4. Exit.

Example Matrix:
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

- **Main diagonal elements:** 1 5 9 (All elements distinct)
- **Sum of main diagonal elements:** 15
- **Diagonal traversal:** 1 2 4 7 5 3 6 8 9

2. Theoretical Background & Algorithm Details

1. Main Diagonal Distinctness:

The main diagonal consists of elements $A[i][i]$ for $0 \leq i < n$. A double loop verifies $A[i][i] \neq A[j][j]$ for all $i \neq j$.

2. Diagonal Sum:

Calculated by summing $\sum_{i=0}^{n-1} A[i][i]$.

3. Zig-Zag Diagonal Traversal:

For an $n \times n$ matrix, there are $2n - 1$ anti-diagonals where elements on the k -th diagonal satisfy $i + j = k$. Traversing alternates direction per diagonal line (odd lines move downward-left, even lines move upward-right).

3. Test Case Table

Test Case	Option	Input Matrix	Expected Output	Status
1	Distinct Check	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$	Main diagonal: 1 5 9	

Result: All elements distinct | Pass | | 2 | Diagonal Sum |

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ | Sum = $1 + 5 + 9 = 15$ | Pass | | 3 | Diagonal

Traversal | $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ | Traversal: 1 2 4 7 5 3 6 8 9 | Pass | | **4**

| Exit | Choice 4 | System Exit | Pass |

4. Source Code Listing

```
#include
#include

void read_matrix(int n, int mat[n][n]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &mat[i][j]);
}

void print_matrix(int n, int mat[n][n]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) printf("%d\t", mat[i][j]);
        printf("\n");
    }
}

void check_distinct_diagonal(int n, int mat[n][n]) {
    bool is_distinct = true;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (mat[i][i] == mat[j][j]) {
                is_distinct = false;
                break;
            }
        }
    }
    if (is_distinct) printf("All main diagonal elements are distinct.\n");
    else printf("Main diagonal elements are NOT distinct.\n");
}

void sum_main_diagonal(int n, int mat[n][n]) {
    int sum = 0;
    for (int i = 0; i < n; i++) sum += mat[i][i];
}
```

```

printf("Sum of main diagonal elements: %d\n", sum);
}

void zigzag_diagonal_traversal(int n, int mat[n][n]) {
    printf("Diagonal traversal: ");
    for (int line = 1; line <= (2 * n - 1); line++) {
        int count = (line < n) ? line : 2 * n - line;
        if (line % 2 != 0) {
            int r = (line - 1 < n) ? line - 1 : n - 1;
            int c = (line - 1 < n) ? 0 : line - n;
            for (int j = 0; j < count; j++) printf("%d ", mat[r - j][c + j]);
        } else {
            int r = (line - 1 < n) ? 0 : line - n;
            int c = (line - 1 < n) ? line - 1 : n - 1;
            for (int j = 0; j < count; j++) printf("%d ", mat[r + j][c - j]);
        }
    }
    printf("\n");
}

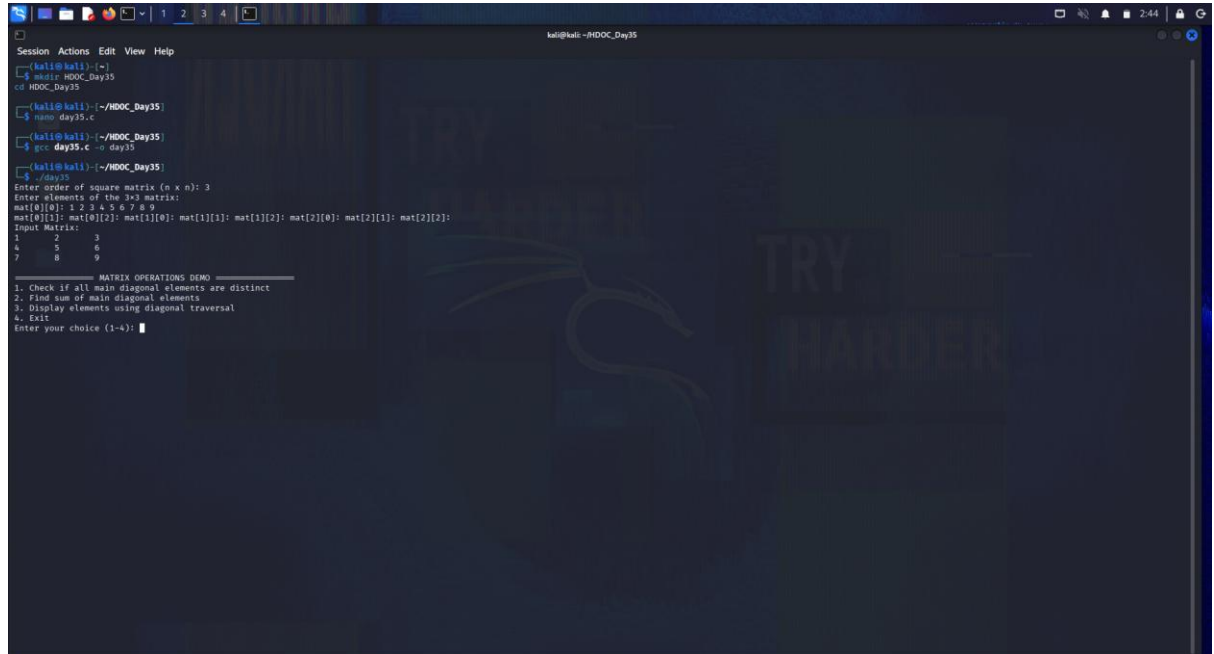
int main(void) {
    int n;
    printf("Enter matrix order n: ");
    scanf("%d", &n);
    int mat[n][n];
    read_matrix(n, mat);

    int choice;
    do {
        printf("\n1. Check Distinct\n2. Diagonal Sum\n3. Diagonal Traversal\n4. Exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: check_distinct_diagonal(n, mat); break;
            case 2: sum_main_diagonal(n, mat); break;
            case 3: zigzag_diagonal_traversal(n, mat); break;
            case 4: printf("Exiting...\n"); break;
        }
    } while (choice != 4);
    return 0;
}

```

}

4. Execution Screenshots



```
kali@kali:~/HDOOC_Day35$ gcc day35.c -o day35
kali@kali:~/HDOOC_Day35$ ./day35
Enter order of square matrix (n x n): 3
Enter elements of the 3x3 matrix:
mat[0][0]: 1 2 3 4 5 6 7 8 9
mat[0][1]: mat[0][2]: mat[1][0]: mat[1][1]: mat[1][2]: mat[2][0]: mat[2][1]: mat[2][2]:
Input Matrix:
1 2 3
4 5 6
7 8 9

===== MATRIX OPERATIONS MENU =====
1. Check if all main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Display elements using diagonal traversal
4. Exit
Enter your choice (1-4):
```